

Understanding
"Attention Is All
You Need"

Agenda

- Natural Language Processing (NLP)
- Introduction
- The Transformer Architecture
- Advantages Over Previous Models
- Training the Transformer
- Conclusion

Natural Language Processing (NLP)

Natural Language Processing (NLP)

- Natural language processing (NLP) is a branch of artificial intelligence that focuses on enabling computers to understand, interpret, and generate human language.
- Also known as computational linguistics, NLP combines advanced machine learning algorithms with linguistic rules to process written and spoken languages.
- The goal of NLP is to achieve human-level language understanding, allowing software to “read” enormous volumes of documents, analyze sentiment and topics, generate summaries, translate between languages, and even create original text.

Natural Language Processing (NLP)

- NLP can be broadly divided into two main categories:
 - Natural language understanding (NLU)
 - Natural language generation (NLG)

Natural language understanding (NLU)

- Natural language understanding is how AI makes sense of text or speech. The word “understand” is a bit of a misnomer because computers don’t inherently understand anything; rather, they can process inputs in a way that leads to outputs that make sense to humans.
- Language is notoriously difficult to describe thoroughly. Even if you manage to document all the words and rules of the standard version of any given language, there are complications such as dialects, slang, sarcasm, context, and how these things change over time.

Natural language generation (NLG)

- Recently, computers' ability to create language is getting much more attention. In fact, the text part of generative AI is a form of natural language generation.
- Today's NLG is essentially a very sophisticated guessing game. Rather than inherently understanding the rules of grammar, generative AI models spit out text a word at a time through probabilistic models that consider the context of their response.
- Because today's large language models (LLMs) have been trained on so much text, their output generally comes across as good human speech, even if sometimes the content is off. (More on that later.)

Introduction

What Problem Does the Paper Solve?

Introduction

- In 2017, Vaswani et al. published a groundbreaking paper titled "Attention Is All You Need" at the Neural Information Processing Systems (NIPS) conference. The paper introduced the Transformer architecture, a new neural network model for natural language processing (NLP) tasks that relies solely on attention mechanisms to process input sequences.
- It is considered a foundational paper in modern artificial intelligence, and a main contributor to the AI boom, as the transformer approach has become the main architecture of a wide variety of AI, such as large language models.

Introduction

- At the time, the focus of the research was on improving Seq2seq techniques for machine translation, but the authors go further in the paper, foreseeing the technique's potential for other tasks like question answering and what is now known as multimodal generative AI.
- As of 2025, the paper has been cited more than 173,000 times, placing it among the top ten most-cited papers of the 21st century.

What Problem Does the Paper Solve?

- Before this paper, most NLP models were based on Recurrent Neural Networks (RNNs), Long Short-Term Memory (LSTM) networks and gated RNNs. While these models worked well for sequential data like text, they had significant drawbacks:
 - Limited parallelism: RNNs process data sequentially, which makes them slow for long sequences.
 - Difficulty capturing long-range dependencies: As sequences grow, it becomes harder for RNNs to relate distant words.

What Problem Does the Paper Solve?

- The authors of “Attention Is All You Need” proposed the Transformer, a model that relies entirely on the mechanism of attention without using any recurrence or convolution.
- This model efficiently handles long-range dependencies, can be trained faster, and processes input in parallel, making it groundbreaking for modern AI.

The Transformer Architecture

What Makes It Different?

Encoder-Decoder Structure

The Transformer architecture consists of two main parts:

Encoder: Processes the input sequence.

Decoder: Produces the output sequence.

Each part contains multiple layers and those layers have sub-layers, and both the encoder and decoder use a mechanism called self-attention to process their respective inputs.

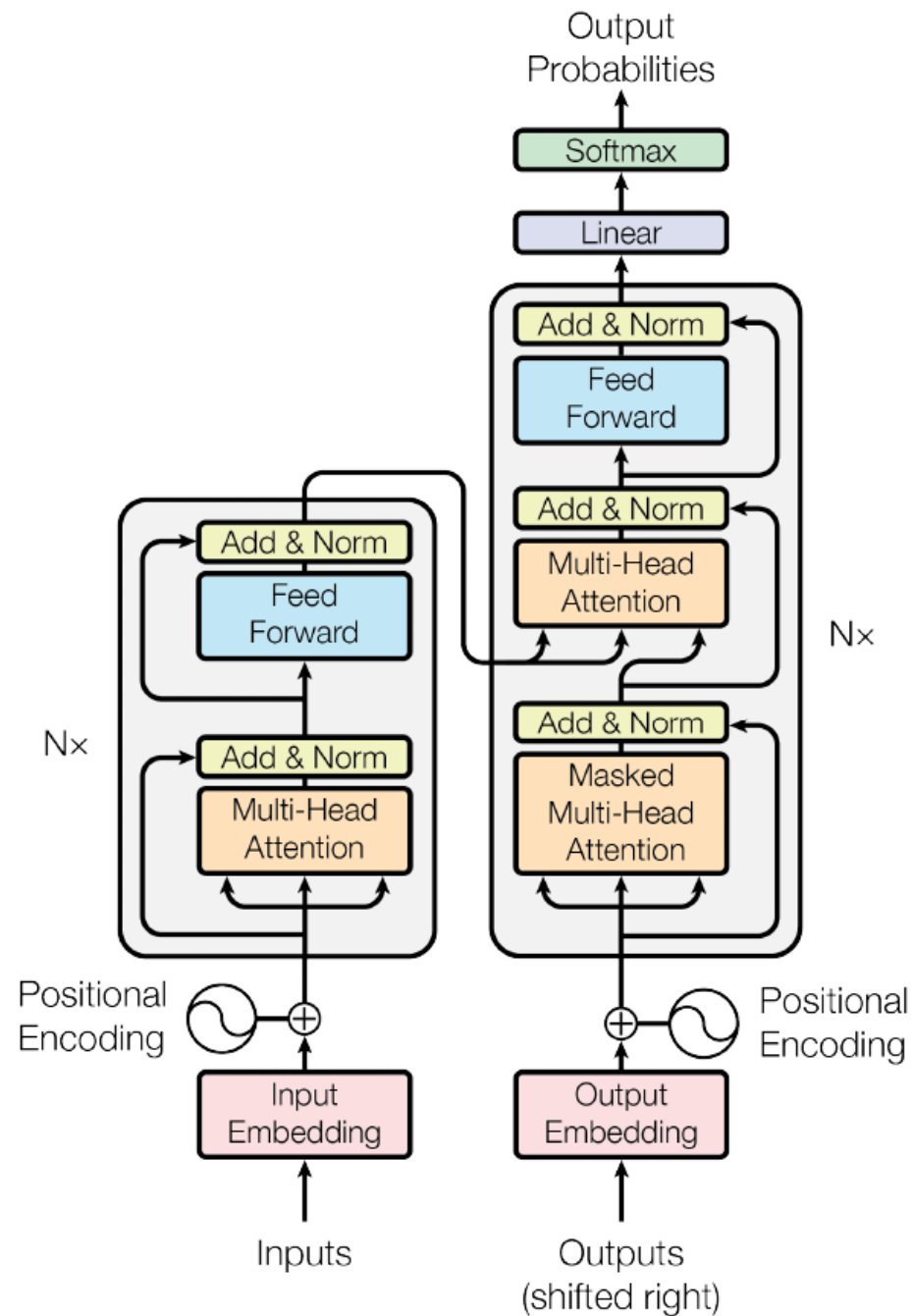
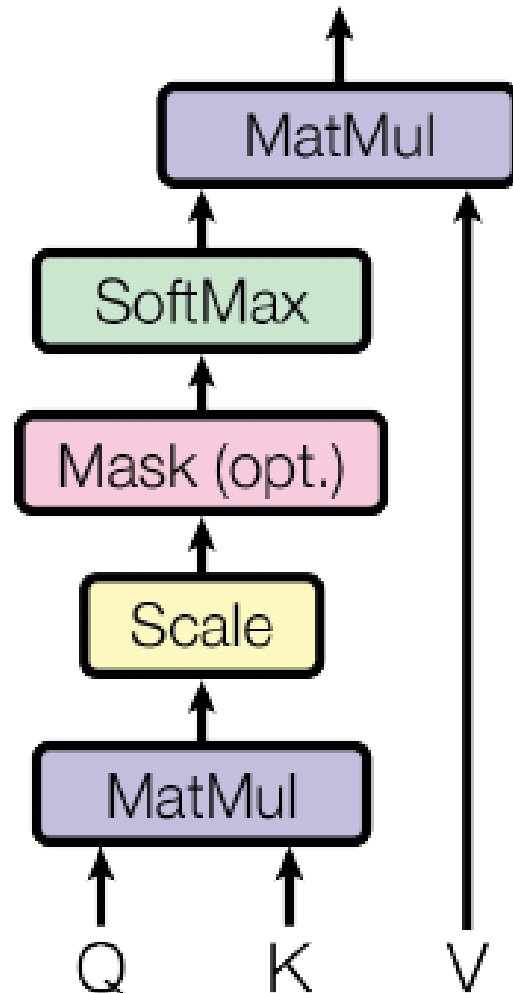


Figure 1: The Transformer - model architecture.

Scaled Dot-Product Attention



Scaled Dot-Product Attention

- Self-attention is the core idea of this model. Instead of processing data sequentially (like RNNs), the model looks at all the words in a sentence simultaneously.
- It then determines which words are most relevant to each other at every layer of the network.
- This allows the model to focus more on important parts of the input and capture relationships between distant words, something RNNs struggled with.

Scaled Dot-Product Attention

To break it down:

Query, Key, Value: Every word in the input is represented as a query, key, and value. The self-attention mechanism computes a weighted sum of values based on the similarity of queries and keys, determining how much focus (attention) each word should receive from other words.

Scaled Dot-Product Attention

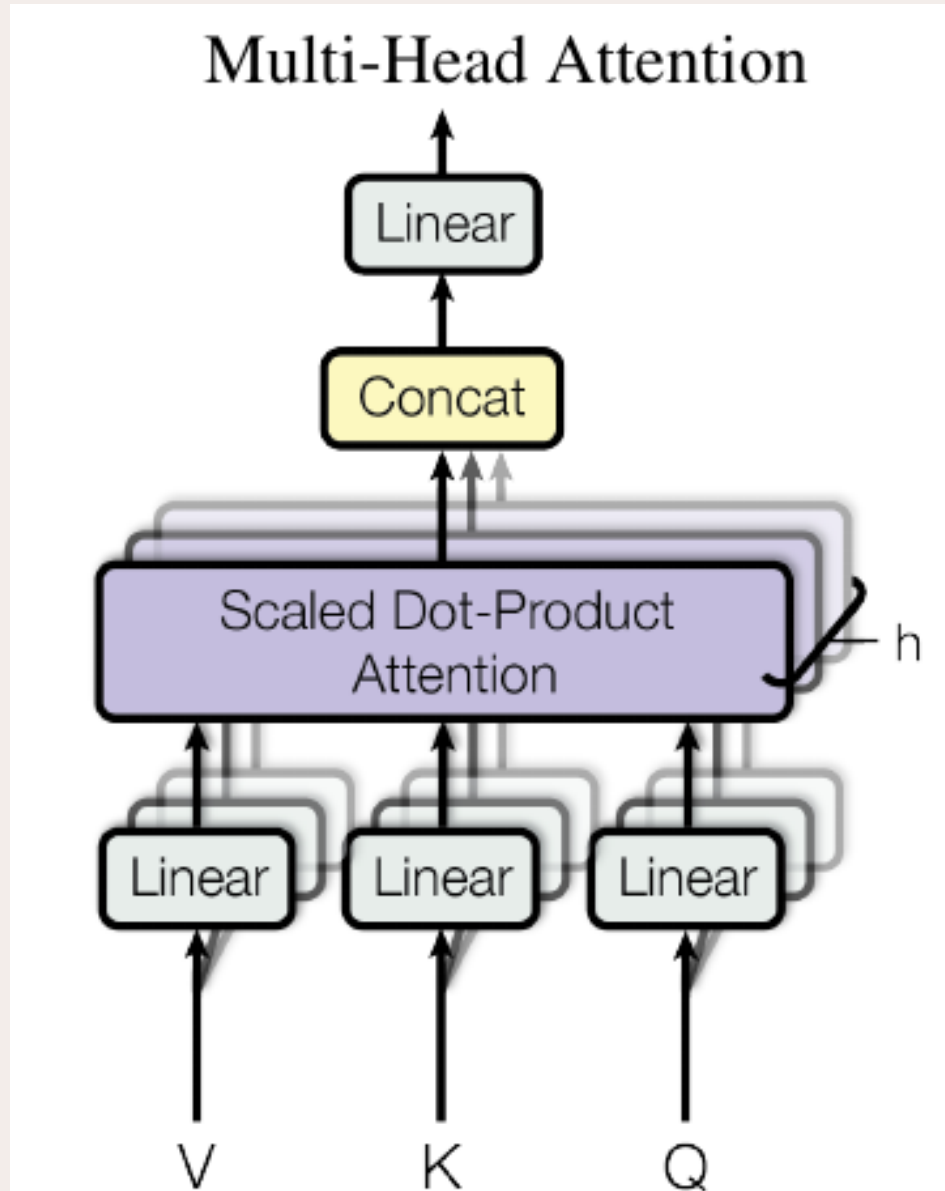
The formula for attention is:

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V$$

Where:

- Q is the query matrix
- K is the key matrix
- V is the value matrix
- d_k is the scaling factor

Multi-Head Attention



- Instead of just one attention calculation, it runs multiple scaled dot-product attentions in parallel, called heads. It doesn't just split the input; it creates different learned projections (like different "views" or "perspectives") of Q , K , and V for each "head."
- Each head focuses on different patterns in the input. Finally, the results from all these parallel heads are combined. This gives the model a much richer understanding of the relationships within the data.

Feed-Forward Networks

After applying self-attention, the model passes the data through a feed-forward neural network (applied separately to each position). These are fully connected layers that help transform the data before sending it to the next layer.

Positional Encoding

- Unlike RNNs, which process data sequentially and maintain a sense of order, Transformers process the entire input simultaneously. To give the model some sense of word order in a sequence, the authors introduced positional encoding. These are added to the word embeddings to provide information about the position of each word in the sentence.
- The formula for positional encoding is:

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$
$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$

Encoder-Decoder Sublayer

A formula for calculating the output of a sublayer of either the encoder or decoder is:

$$\text{Output} = \text{LayerNorm}(x + \text{Sublayer}(x))$$

Where:

x is the input to the sublayer.

$\text{Sublayer}(x)$ is the actual computation (e.g., attention, feed-forward network).

$x + \text{Sublayer}(x)$ is called a residual connection, a fancy term for adding the input back to the output of the sublayer. It's crucial for stable training and optimisation in deep networks.

The Encoder

- This has **6 layers**, and each layer is made up of **2 sublayers**:
 - **Multi-head self-attention**: This means each word looks at all the other words in the sentence, including itself, to figure out which ones matter most. This helps the model understand the full context of the input.
 - **Feed-forward network (FFN)**: A mini neural network applied to each word separately.
- The formula $\text{Output} = \text{LayerNorm}(x + \text{Sublayer}(x))$ applies to both of these sublayers with the “Sublayer(x)” function either being the multihead self-attention or the FFN.

The Decoder

- The decoder also has **6 layers**, but with **3 sublayers** per layer:
 - **Masked multi-head self-attention**: Just like the encoder's, but with a small twist: masking. Each word can only look at itself and the ones before it, not the ones ahead. This way, the model doesn't cheat by seeing future words when generating a sentence.
 - **Cross-attention**: This allows the decoder to look at the output of the encoder. This is how the input sequence (e.g., the English sentence) informs the generation of the output sequence (e.g., the German translation).
 - **Feed-forward network (FFN)**: A mini neural network applied to each word separately.

The Decoder

- The mathematical representation for one layer of the decoder would be:
 - $z1 = \text{LayerNorm}(x + \text{MultiHeadMaskedSelfAttention}(x))$
 - $z2 = \text{LayerNorm}(z1 + \text{MultiHeadCrossAttention}(z1, \text{EncoderOutput}))$
 - $z3 = \text{LayerNorm}(z2 + \text{FeedForward}(z2))$
- With $z3$ being the output of the layer. As you can see, it's still built on the general formula.

Advantages Over Previous Models

Advantages Over Previous Models

- The Transformer model brings several advantages compared to previous models like RNNs and LSTMs:
 - Parallelism: Self-attention allows the model to process all words in the input simultaneously, speeding up training significantly.
 - Long-Range Dependencies: The attention mechanism helps capture relationships between words, no matter how far apart they are in a sequence.
 - Scalability: Since the model doesn't depend on sequential processing, it's easier to scale to longer texts and bigger datasets.

Training the Transformer

How It Works

Training Data and Batching

- The authors trained the Transformer model on machine translation tasks, specifically translating between English and German, and English and French.
- The English-to-German translation model was trained on the 2014 WMT (Workshop on Statistical Machine Translation) English-German dataset, consisting of nearly 4.5 million sentences derived from TED Talks and high-quality news articles.
- A separate translation model was trained on the much larger 2014 WMT English-French dataset, consisting of 36 million sentences. Both datasets were encoded with byte-pair encoding.

Hardware and Schedule

- The models were trained using 8 NVIDIA P100 GPUs.
- The base models were trained for 100,000 steps and the big models were trained for 300,000 steps - each step taking about 0.4 seconds to complete for the base models and 1.0 seconds for the big models.
- The base model trained for a total of 12 hours, and the big model trained for a total of 3.5 days. Both the base and big models outperforms the 2017 state-of-the-art in both English-German and English-French while achieving the comparatively lowest training cost.

Hyperparameters and regularization

- For training the Transformer model, the authors used the Adam optimizer and increased the learning rate linearly for the first 4000 (warmup) steps and decreased it proportionally to inverse square root of the current step number.
- Dropout layers were applied to the output of each sub-layer before normalization, the sums of the embeddings, and the positional encodings. The dropout rate was set to 0.1.
- Label smoothing was applied with a value of 0.1 which "improves accuracy and BLEU score"

Results

Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (big)	28.4	41.8	$2.3 \cdot 10^{19}$	

Conclusion

Why Does It Matter?

Conclusion

“Attention Is All You Need” is a landmark paper in the field of artificial intelligence. By moving away from sequential processing and focusing entirely on attention mechanisms, the Transformer model solved many of the bottlenecks faced by previous architectures. Its impact on NLP and machine learning continues to grow, with new models and innovations building on its foundations every year.

Thank
you

Aashish Mahato